

# Web scraping and Social Media Scraping

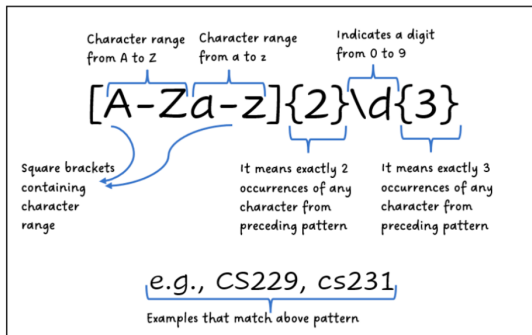
regex

Maciej Świtła, PhD

Ewa Weychert, PhD

# Aim of today's class

- Basics of writing regular expressions
- Basics of reading regular expressions



# Presentation Outline

- 1 Introduction to regular expressions
- 2 Regex Basics
- 3 Character Classes
- 4 Metacharacters
- 5 Quantifiers
- 6 Combining Regex Elements
- 7 Anchors, Alternation, Grouping in Regex
- 8 Types of Regex - summary
- 9 Regex in python - re module

# Definition of Regular Expressions (Regex)

- As the old computer-science joke goes: *“Let’s say you have a problem, and you decide to solve it with regular expressions. Well, now you have two problems.”*
- Regular expressions (regex) are often taught using large tables of symbols, which can make them look confusing. For example: `^r(ab|ac)` (matches `rab` or `rac` only at the beginning of a string).
- Because of this, many programmers avoid regex and write unnecessarily complicated searching or filtering functions.
- In reality, many problems can be solved with a single line of regex.
- Regular expressions are used to identify patterns in strings.

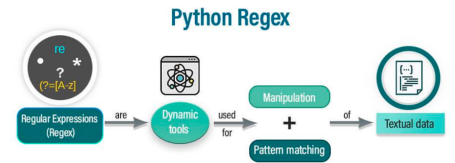
## Where would you use regex?

By using regular expressions and their metacharacters, you can perform various tasks such as:

- validating input
- extracting specific information from a text → web scraping
- replacing patterns → text mining
- and much more

Regular expressions are widely used in:

- text processing
- data validation
- web scraping
- other textual data applications



## Example: Selecting Names with Regex

**Goal:** Identify Polish female names that typically end with the letter a.

Example list of names:

```
["Ala", "Krzysztof", "Paweł", "Alina"]
```

Using a regular expression, we can select only the names that match the pattern of ending with the letter a.

Result:

```
["Ala", "Alina"]
```

**Idea:** Regex allows us to filter text elements based on patterns, such as words that end with a specific letter.

## What is a Regular String?

A regular string is any string that can be generated by a series of simple rules.

Example rules:

- 1 Write the letter a at least once.
- 2 Append the letter b exactly five times.
- 3 Append the letter c any even number of times.
- 4 Optionally add the letter d at the end.

Example valid strings:

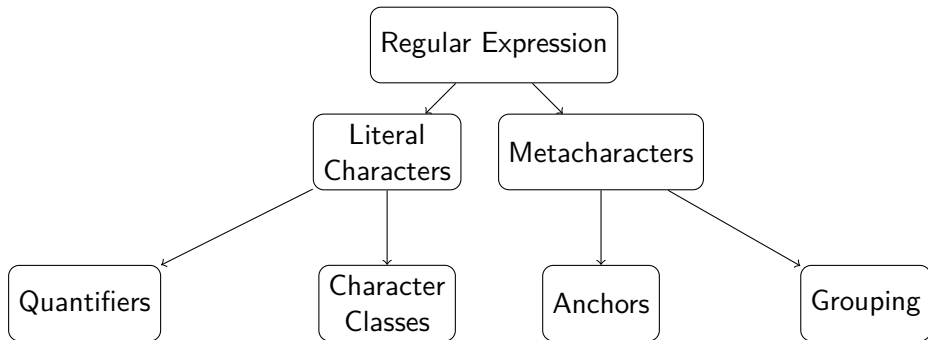
- aaaabbbbbcccd
- aabbbbbcc

Regular expressions are simply a \*\*short way to describe these rules\*\*.

Example regex:

$$aa^* bbbbb(cc)^*(d \mid)$$

## Types of Regex – Summary





## Regex Components

A regular expression (regex) pattern is built from two types of elements:

- **Literal characters** – characters that match themselves exactly.
- **Metacharacters** – special symbols that define matching rules or patterns.

### Examples of literal characters

- `cat` → matches the exact word `cat`
- `dog` → matches the exact word `dog`

### Examples of metacharacters

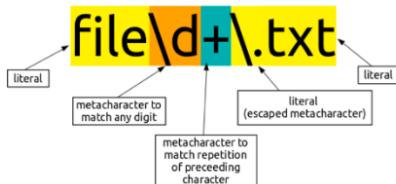
- `[0-9]` → matches any digit from 0 to 9
- `[a-z]` → matches any lowercase letter

### Example text

I have 2 cats and 1 dog

- Literal pattern `cat` matches `cat`
- Metacharacter pattern `[0-9]` matches `2` and `1`

## Example: Regex for File Names



So, the regular expression we need here is:

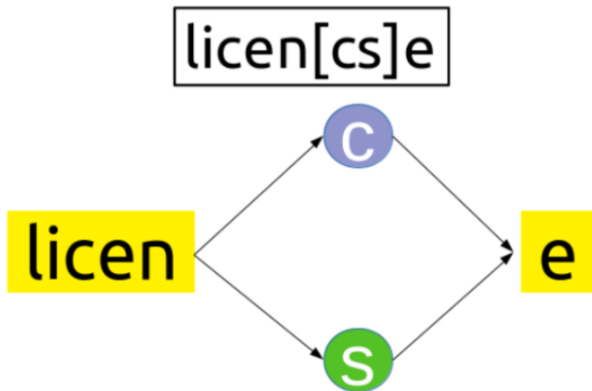
```
file\d+\.txt
```

This expression can be understood as follows:

- `file` is a substring of literals which are matched with the input as it is.
- `\d` is a metacharacter which instructs the software to match this position with a digit (0-9).
- `+` is also a metacharacter which instructs the software to match one or more iterations of the preceding character (`\d` in this case).
- `\.` is a literal. `.` is a metacharacter but we want to use it as a literal in this case. Hence, we escape it using `\` character.
- `txt` is a substring of literals which are matched with the input as it is.

# Character Class

Yesterday, I was driving my car without a driving **licence**. The traffic police stopped me and asked me for my **license**. I told them that I forgot my **licence** at home.



# Character Classes

Examples:

- `[0-9]` matches digits
- `[a-z]` matches lowercase letters
- `[A-Z]` matches uppercase letters
- `[aeiou]` matches vowels

## Metacharacters Basics

Metacharacters are special symbols in regular expressions that have predefined meanings. They allow you to specify rules and constraints for pattern matching.

| Nr | Metacharacter | Definition                                                            |
|----|---------------|-----------------------------------------------------------------------|
| 1  | .             | Matches any single character except a newline.                        |
| 2  | ^             | Matches the start of a string.                                        |
| 3  | \$            | Matches the end of a string.                                          |
| 4  | *             | Matches zero or more occurrences of the preceding character or group. |
| 5  | +             | Matches one or more occurrences of the preceding character or group.  |
| 6  | ?             | Matches zero or one occurrence of the preceding character or group.   |
| 7  |               | Acts as an OR operator, allowing multiple patterns.                   |

# Quantifiers

|       |                       |
|-------|-----------------------|
| *     | Zero or more times    |
| +     | One or more times     |
| ?     | Zero or one time      |
| {n}   | Exactly n times       |
| {n,m} | Between n and m times |

Example:

- `a+` matches one or more a
- `[0-9]{3}` matches exactly three digits

# Regular Expressions: Literal, Classes, Quantifiers

## Literal Characters

- In regular expressions, literal characters refer to specific characters matched exactly as they appear.
- For example, the pattern `cat` will only match the sequence of letters `cat` in the given text.
- Literal characters allow you to create precise matches in regular expressions.
- Example: To match the word `hello`, use the pattern `hello`.

## Character Classes

- Character classes define a set of characters that can match a single character in the text.
- They are enclosed within square brackets `[ ]`.
- Example: The pattern `[aeiou]` matches any vowel.
- Example: The regex pattern `[0-9]` matches any digit.

## Quantifiers

- Quantifiers control how many times a character or group can occur.
- They define repetition rules for pattern matching.
- Example: `a+` matches one or more occurrences of the letter `a`.

# Anchors, Alternation, Grouping in Regex

## Anchors

- Anchors in regular expressions match positions rather than characters.
- They specify where a pattern should start or end in the text.
- The caret symbol `^` represents the start of a string, and the dollar symbol `$` represents the end of a string.
- Example: The pattern `^Hello` matches the word `Hello` only if it appears at the beginning of a line.

## Alternation

- Alternation is represented by the vertical bar `|`. It allows you to specify multiple alternatives for matching.
- The regex engine matches either the expression before or after the vertical bar.
- Example: The pattern `cat|dog` matches either `cat` or `dog`.

## Grouping and Capturing

- Grouping is done using parentheses `( )`.
- It allows patterns to be treated as logical units and used with quantifiers or alternations. Capturing groups store parts of the matched text for later use.
- Example: The pattern `(ab)+` matches one or more occurrences of the



## Types of Regex - summary

- **Literal character:** A literal character is the most basic regular expression. It matches the exact character written. Example: To match the letter `r`, the pattern is simply `r`.
- **Metacharacter:** Metacharacters tell the regex engine that the following character has a special meaning. They are usually preceded by a backslash `\`. Examples include `^` (start of line), `$` (end of line), and `.` (any character).
- **Character class:** A character class tells the engine to match one character from a list of possible characters. It is written using square brackets `[ ]`. Example: `[abc]` matches `a`, `b`, or `c`.
- **Capture group:** A capture group is created using round parentheses `( )`. They allow parts of a pattern to be grouped together and used with other regex features such as quantifiers.

# Regular Expressions: Regexes in Python

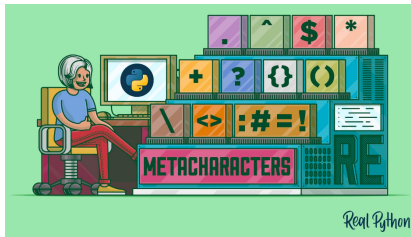
## re module in Python

Regex functionality in Python resides in a module named `re`. The `re` module contains many useful functions and methods for working with regular expressions.

Most of these functions will be introduced in the next tutorials.

## Importing the module:

```
import re
```



## Most Common Functions in re Module

- `re.compile(pattern)` – Compile a regular expression pattern into a regular expression object.
- `re.search(pattern, string)` – Scan through a string looking for the first location where the regular expression pattern produces a match and return a corresponding Match object. Returns None if no position in the string matches the pattern.
- `re.split(pattern, string)` – Split a string by the occurrences of a pattern. If capturing parentheses are used in the pattern, the text of all groups is also returned as part of the resulting list.
- `re.findall(pattern, string)` – Return all non-overlapping matches of the pattern in a string as a list of strings or tuples. The string is scanned left-to-right and matches are returned in the order found.

## Most Common Functions in re Module

- `re.sub(pattern, repl, string)` – Return the string obtained by replacing the leftmost non-overlapping occurrences of the pattern in a string with the replacement text `repl`. If the pattern is not found, the string is returned unchanged.
- `match()` – Determine if the regular expression matches at the beginning of the string.
- `search()` – Scan through a string, looking for any location where the regular expression matches.
- `findall()` – Find all substrings where the regular expression matches and return them as a list.
- `finditer()` – Find all substrings where the regular expression matches and return them as an iterator.

## Regex in Python - example

Regex functionality is available in the re module.

### Example

```
pattern = r"\d+" text = "There are 24 apples"  
result = re.findall(pattern, text)  
print(result)  
['24']
```

## References

- <https://developers.google.com/edu/python/regular-expressions?hl=pl>
- <https://www.programiz.com/python-programming/regex>
- <https://docs.python.org/3/library/re.html>
- <https://www.datacamp.com/cheat-sheet/regular-expresso>
- <https://pub.towardsai.net/regular-expression-regex-in-python-the-basics-b8f2cd041bd>
- <https://pub.towardsai.net/regular-expressions-regex-in-python-advanced-concepts-69b>
- <https://cheatography.com/davechild/cheat-sheets/regular-expressions/>
- <https://github.com/nikhilkumarsingh/RegEx-In-Python/blob/master/01.%20Introduction.ipynb>
- Mitchell, R. (2024). *Web Scraping with Python*. O'Reilly Media, Inc. Chapter 2.